

DevOps and Automations TASK – 4

1.Install Terraform on your system and write a main.tf file that provisions a single AWS EC2 instance with the tag Name = 'MusicApp-Server'.

Ans :

- Install Terraform from the official HashiCorp page:
- Terraform installation
- Verify the installation:
- terraform --version
- Create a project folder:
- mkdir MusicApp-Terraform
- cd MusicApp-Terraform
- Create main.tf:
- notepad main.tf
- Copy this configuration into main.tf:
- terraform {
- required_providers {
- aws = {
- source = "hashicorp/aws"
- }
- }
- }
- }
- provider "aws" {

- region = "us-east-1"
- }
- resource "aws_instance" "music_app_server" {
- ami = "ami-0c02fb55956c7d316"
- instance_type = "t2.micro"
- tags = {
- Name = "MusicApp-Server"
- }
- }
- Save and close main.tf.
- Initialize Terraform:
- terraform init
- Format the configuration:
- terraform fmt
- Validate it:
- terraform validate
- Expected result:
- Success! The configuration is valid.

2.Create a simple Node.js application that displays 'Welcome to My Playlist' on port 3000, and add a Dockerfile to containerize it.

Ans :

- Create the folder
- mkdir playlist-app
- cd playlist-app

- Create package.json
- notepad package.json
- Paste:
 - {
 - "name": "playlist-app",
 - "version": "1.0.0",
 - "description": "Simple playlist Node.js application",
 - "main": "app.js",
 - "scripts": {
 - "start": "node app.js"
 - },
 - "dependencies": {
 - "express": "^5.1.0"
 - }
 - }
- Save and close.
- Create app.js
- notepad app.js
- Paste:
 - const express = require("express");
 -
 - const app = express();
 - const PORT = 3000;
 -
 - app.get("/", (req, res) => {
 - res.send("Welcome to My Playlist");
 - });
 -
 - app.listen(PORT, () => {

- `console.log(`Playlist app running on port ${PORT}`);`
 - `});`
 - Save and close.
 - Create the Dockerfile
 - notepad Dockerfile
 - Paste:
 - FROM node:22-alpine
-
- WORKDIR /app
-
- COPY package*.json ./
-
- RUN npm install
-
- COPY app.js .
-
- EXPOSE 3000
-
- CMD ["npm", "start"]
 - Save and close.
 - Build the Docker image
 - Make sure Docker Desktop is running, then execute:
 - `docker build -t playlist-app .`
 - Run the container
 - `docker run -d --name playlist-container -p 3000:3000 playlist-app`
 - Test the application
 - Open your browser:

- `http://localhost:3000`
- You should see:
- Welcome to My Playlist
- Verify the container
- `docker ps`

3. Set up a GitHub Actions workflow file (.github/workflows/deploy.yml) that triggers on every push to main, runs 'terraform init' and 'terraform apply' to provision your AWS EC2 instance, and then builds your Docker image.
Hint: Use the 'hashicorp/setup-terraform' and 'actions/checkout' actions in your workflow.

Ans :

- Create the folders
- From your project directory:
- `mkdir .github`
- `mkdir .github\workflows`
- Create `deploy.yml`
- `notepad .github\workflows\deploy.yml`
- Copy this workflow
- name: Deploy Music App
- on:
- push:
- branches:
 - main
- permissions:
- contents: read

- jobs:
- deploy:
- runs-on: ubuntu-latest

- steps:
 - name: Checkout repository
- uses: actions/checkout@v7

- name: Configure AWS credentials
- uses: aws-actions/configure-aws-credentials@v6
- with:
 - aws-access-key-id: \${{ secrets.AWS_ACCESS_KEY_ID }}
 - aws-secret-access-key: \${{ secrets.AWS_SECRET_ACCESS_KEY }}
 - aws-region: us-east-1

- name: Setup Terraform
- uses: hashicorp/setup-terraform@v4

- name: Terraform Init
- run: terraform init

- name: Terraform Apply
- run: terraform apply -auto-approve

- name: Build Docker image

- `run: docker build -t music-playlist-app .`
- The AWS credentials are referenced through GitHub Secrets rather than being written into the workflow. For production use, AWS recommends short-lived OIDC credentials instead of long-lived access keys.
- Add AWS credentials to GitHub
- In your GitHub repository:
 - Settings → Secrets and variables → Actions → New repository secret
 - Create:
 - `AWS_ACCESS_KEY_ID`
 - Value:
 - `YOUR_AWS_ACCESS_KEY`
 - Create another:
 - `AWS_SECRET_ACCESS_KEY`
 - Value:
 - `YOUR_AWS_SECRET_KEY`
- Do not put the actual credentials directly inside `deploy.yml`.
- Commit the workflow
 - `git add .github/workflows/deploy.yml`
 - `git commit -m "Add GitHub Actions deployment workflow"`
 - `git push origin main`
- Check the workflow
- Go to your GitHub repository and click:
 - Actions
 - You should see:
 - Deploy Music App
 - Click the workflow run.
 - You should see these steps:
 - ✓ Checkout repository
 - ✓ Configure AWS credentials

- ✓ Setup Terraform
- ✓ Terraform Init
- ✓ Terraform Apply
- ✓ Build Docker image

**4.Modify your pipeline so that after the EC2 instance is provisioned, your Node.js Docker container is deployed to the instance automatically.

Hint: You may use SSH steps in GitHub Actions to connect and run Docker commands on the instance.**

Ans :

- Update outputs.tf
- Add the EC2 public IP:
- output "ec2_public_ip" {
- description = "Public IP address of the EC2 instance"
- value = aws_instance.music_app_server.public_ip
- }
- If your resource has a different name, use the correct resource reference.
- Make sure the EC2 instance allows SSH and port 3000
- Your security group should allow:
- SSH TCP 22
- App TCP 3000
- For real deployments, restrict SSH to your own IP instead of 0.0.0.0/0.
- Update .github/workflows/deploy.yml
- Use this complete workflow:
- name: Deploy Music App
- on:

- push:
- branches:
 - main
- permissions:
- contents: read
- jobs:
- deploy:
- runs-on: ubuntu-latest
- steps:
 - name: Checkout repository
- uses: actions/checkout@v7
- steps:
 - name: Configure AWS credentials
- uses: aws-actions/configure-aws-credentials@v6
- with:
 - aws-access-key-id: \${ secrets.AWS_ACCESS_KEY_ID }
 - aws-secret-access-key: \${ secrets.AWS_SECRET_ACCESS_KEY }
 - aws-region: us-east-1
- steps:
 - name: Setup Terraform
- uses: hashicorp/setup-terraform@v4

- name: Terraform Init
- run: terraform init

- name: Terraform Apply
- run: terraform apply -auto-approve

- name: Get EC2 public IP
- id: terraform
- run: |
 - echo "EC2_IP=\$(terraform output -raw ec2_public_ip)" >> "\$GITHUB_ENV"

- name: Deploy Docker container to EC2
- uses: appleboy/ssh-action@v1
- with:
 - host: \${ env.EC2_IP }
 - username: ec2-user
 - key: \${ secrets.EC2_SSH_PRIVATE_KEY }
 - port: 22
 - script: |
 - set -e
 - cd ~/music-app || mkdir -p ~/music-app
 - cd ~/music-app
 - git clone https://github.com/\${ github.repository }.git
 - cd app

- `docker stop music-playlist-app || true`
- `docker rm music-playlist-app || true`

- `docker build -t music-playlist-app .`
- `docker run -d \`
 - `--name music-playlist-app \`
 - `-p 3000:3000 \`
 - `music-playlist-app`
- The appleboy/ssh-action supports connecting with an SSH host, username, port, and private key, then executing commands remotely. G GitHub
- Add the SSH private key to GitHub
- In your GitHub repository go to:
- Settings → Secrets and variables → Actions → New repository secret
- Create:
- EC2_SSH_PRIVATE_KEY
- Paste the private SSH key that corresponds to the public key installed on your EC2 instance.
- Keep the private key in GitHub Secrets; don't put it directly into the workflow. GitHub provides repository/environment secrets specifically for this kind of sensitive value.
- You should already have:
- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- EC2_SSH_PRIVATE_KEY
- Make sure Docker is installed on EC2
- The EC2 instance must have Docker installed and the ec2-user must be able to run Docker.

- For Amazon Linux, you can install it manually once:
- `sudo dnf install -y docker`
- `sudo systemctl enable --now docker`
- `sudo usermod -aG docker ec2-user`
- Then log out and back in.
- Commit and push
- From your project folder:
- `git add .`
- `git commit -m "Deploy Docker app to EC2"`
- `git push origin main`
- Check GitHub Actions
- Go to your repository → Actions → Deploy Music App.
- The workflow should perform:
- Checkout
- ↓
- Configure AWS
- ↓
- Terraform Init
- ↓
- Terraform Apply
- ↓
- Get EC2 Public IP
- ↓
- SSH into EC2
- ↓
- Clone application
- ↓
- Build Docker image
- ↓
- Run Docker container
- Verify the application

- After the workflow succeeds, open:
- `http://YOUR_EC2_PUBLIC_IP:3000`

5. Use ChatGPT to generate a Terraform resource block for creating an AWS S3 bucket named 'my-playlist-app-bucket', then add it to your main.tf and apply the changes.

Ans :

- Open your Terraform project
- `cd C:\music-streaming-server`
- Open main.tf
- notepad main.tf
- Add this S3 resource block
- `resource "aws_s3_bucket" "playlist_app_bucket" {`
- `bucket = "my-playlist-app-bucket"`
- `tags = {`
- `Name = "my-playlist-app-bucket"`
- `Owner = "YOUR_NAME"`
- `}`
- `}`
- Replace YOUR_NAME with your name.
- Your main.tf should contain your existing AWS provider configuration plus this resource.
- Save main.tf
- Close Notepad after saving.
- Format the Terraform configuration
- `terraform fmt`
- Validate the configuration
- `terraform validate`
- You should see:

- Success! The configuration is valid.
- Preview the change
- terraform plan
- Look for:
- aws_s3_bucket.playlist_app_bucket
- and:
- Plan: 1 to add, 0 to change, 0 to destroy.
- Apply the change
- terraform apply
- When Terraform asks for confirmation, enter:
- yes